

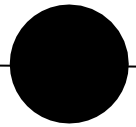
React e React Native

Desenvolvimento para Dispositivos Móveis

Prof. Me. Jackson Meires
jackson.meires@ifsc.edu.br

Sumário

- ◉ Introdução
 - ◉ Fundamentos
 - ◉ Arquitetura
- ◉ JSX
- ◉ Elementos
- ◉ Componentes
 - ◉ Stateless e Stateful
- ◉ Props
- ◉ State
- ◉ Styles
- ◉ Instalação
 - ◉ Execução do projeto
- ◉ Exercícios



Introdução

O que é o React?

“Uma biblioteca Javascript para construção de interfaces de usuário web”

<https://pt-br.react.dev/learn>

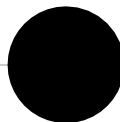
ou

<https://www.w3schools.com/react/>

“Learn once, write anywhere!”



<https://reactnative.dev/>



O que é o React Native?

- Uma biblioteca criada pelo Facebook em 2015, usando *Javascript* e *React*, possibilitando desenvolver apps mobile multiplataforma
- Permite o desenvolvimento de apps multiplataforma (Android e iOS) com JavaScript utilizando um único código-fonte
- Utiliza a mesma sintaxe do ReactJS, facilitando o aprendizado para quem já conhece a biblioteca
- Grande comunidade de desenvolvedores e ampla variedade de bibliotecas e recursos disponíveis

Fundamentos

- Combina a biblioteca *React* com APIs nativas do Android e iOS
- Permite a criação de interfaces de usuário interativas e responsivas.
- Suporta diversos componentes prontos para uso, como botões, inputs, listas e imagens
- Possibilita a integração com *APIs* nativas e bibliotecas de terceiros

Fundamentos

Declarativo

Facilita a criação de interfaces interativas. Para cada estado da aplicação, atualiza e renderiza somente os componentes necessários à medida que os dados mudam

Baseado em Componentes

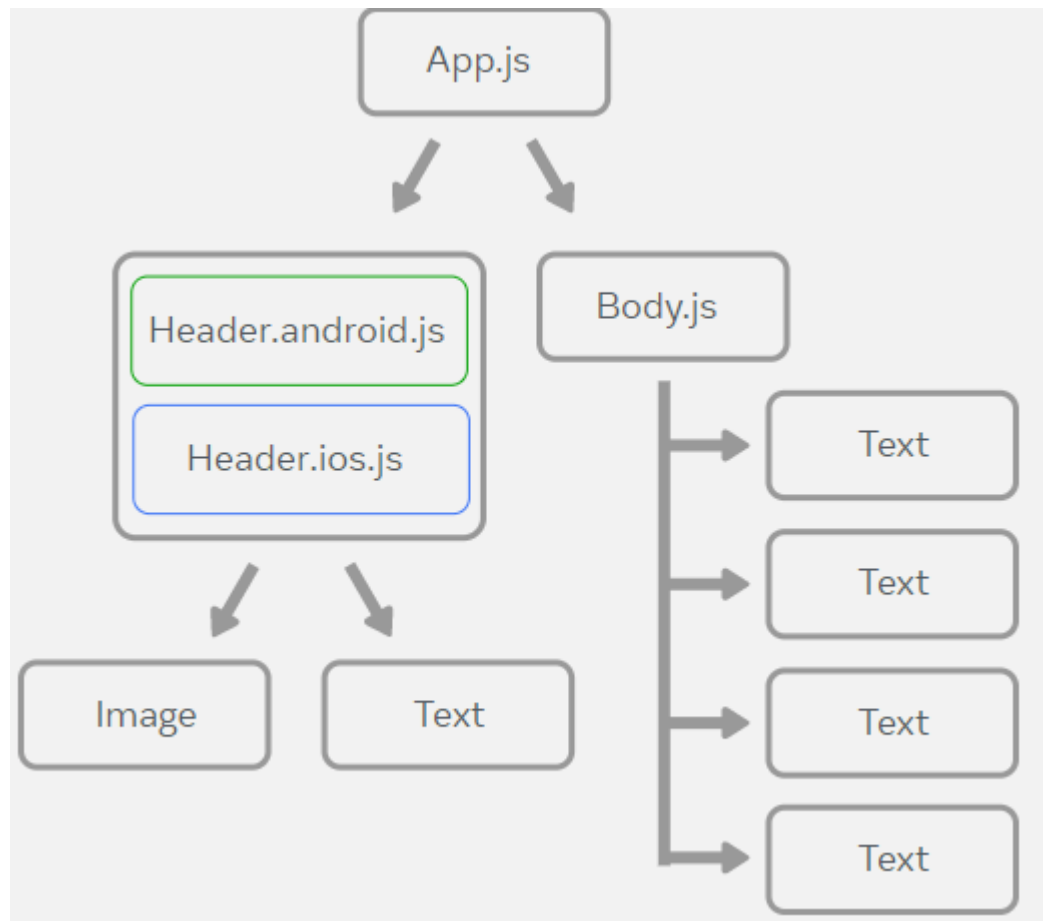
Utiliza o conceito de componentes encapsulados que gerenciam seu próprio estado

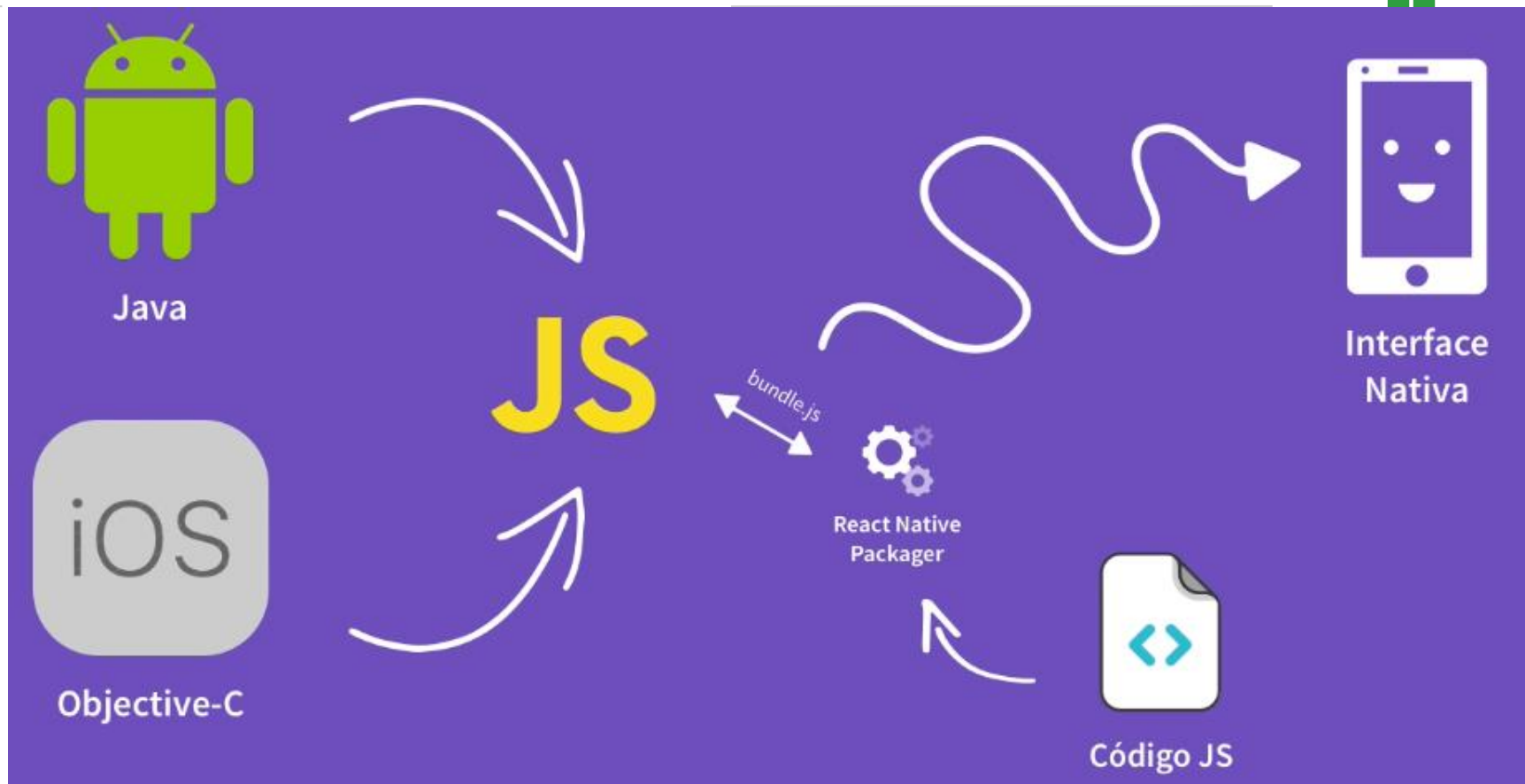
Utilizado em Diversos Lugares

Pode ser renderizado no servidor, usando Node, e em aplicações móveis através do React Native

Arquitetura

- Arquitetura em camadas, separando a lógica da interface do usuário da lógica específica de cada plataforma
- Ponte *JavaScript* que permite comunicação entre o código *JavaScript* e o código nativo
- Threads separadas para *JavaScript* e código nativo, garantindo performance e responsividade

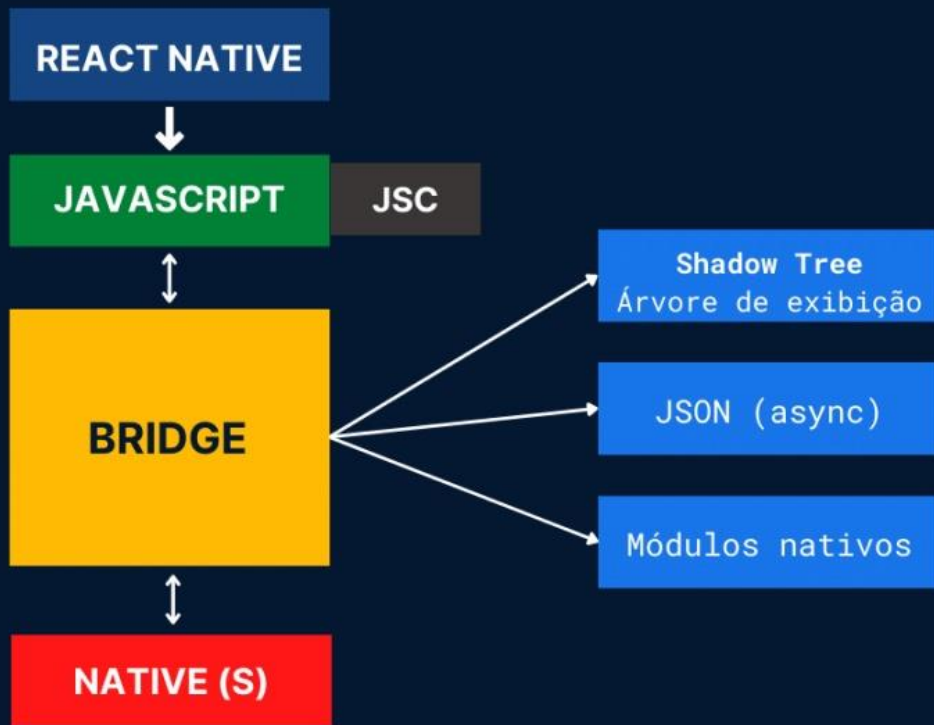




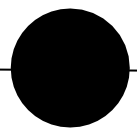


A ARQUITETURA DO REACT NATIVE

Entenda como se dividem as 4 principais seções:



**INSTITUTO
FEDERAL**
Santa Catarina



JSX

JSX

Considere essa declaração de variável:

```
const msg = <Text>Olá Mundo</Text>
```

Essa sintaxe estranha de tags não é uma string, nem HTML. É chamada JSX e é uma extensão de sintaxe para JavaScript. JSX produz “elementos” do React.

```
import { Text, View } from 'react-native';

export default function App() {
  return (
    <View>
      <Text>Bem vindo ao React e React Native</Text>
    </View>
  );
}
```

JSX com filhos.



Utilize chaves para expressões.

```
function Ola({ name }) {  
  return <Text>Hello, {name}</Text>;  
}
```

```
export default function App() {  
  return <Ola name="world" />  
}
```

Utilize aspas duplas para strings literais.

<https://pt-br.react.dev/learn/your-first-component>

JSX com expressões.



```

import { Text, View, Image } from 'react-native';

function Profile() {
  return (
    <Image
      alt="Katherine Johnson"
      style={{ width: 50,height: 50}}
      source={{ uri: 'https://i.imgur.com/MK3eW3As.jpg' }}
      //source={{uri: 'https://reactnative.dev/img/tiny_logo.png',}}
    />
    //duas chaves: A primeira representa o javascript, e a Segunda um objeto no javascript
  );
}

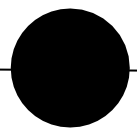
export default function Gallery() {
  return (
    <View>
      <Text>Cientistas incríveis</Text>
      <Profile />
      <Profile />
      <Profile />
    </View>
  );
}

```

Cientistas incríveis



JSX com expressões 2.



Elementos

Elementos

São os menores blocos de construção de aplicativos React e React-Native.

Diferente dos elementos DOM do navegador, elementos React são objetos simples e utilizam menos recursos.

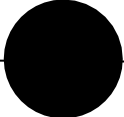
O React é o responsável por exibir os elementos do React-Native

Renderizando Elementos

Para renderizar um elemento na tela basta utilizar o método `App()`.

```
export default function App() {  
  return (  
    <View>  
      <Text>Jackson</Text>  
      <Text>Meires</Text>  
    </View>  
  );  
}
```

Normalmente aplicativos React possuem somente um nó raiz!



Componentes



INSTITUTO
FEDERAL
Santa Catarina

Modularização em forma de componentes

A programação com React Native é praticamente baseada em **componentes**. Isso ajuda muito na **modularização do nosso código** e na parte de **abstração de funções complexas**, além de permitir a **reutilização de código** em diversas aplicações

O site do [NPM](#) contém diversas **bibliotecas** criadas por empresas e comunidades que podem ser integradas em seus projetos como um componente comum. Isso facilita e agiliza muito a criação de aplicativos

React e React Native

Sintaxe

- No React '*clássico*', os elementos não são escritos com as tags **HTML**:

`<div>`, `<span>`, `<ul>`

- No React Native as tags** são elementos que convertem diretamente para linguagem nativa, na tabela abaixo estão alguns exemplos:

React	React Native
<code><div></code>	<code><View></code>
<code></code>	<code><Text></code>
<code></code> , <code></code>	<code><ListView></code>
<code></code>	<code><Image></code>

Componentes

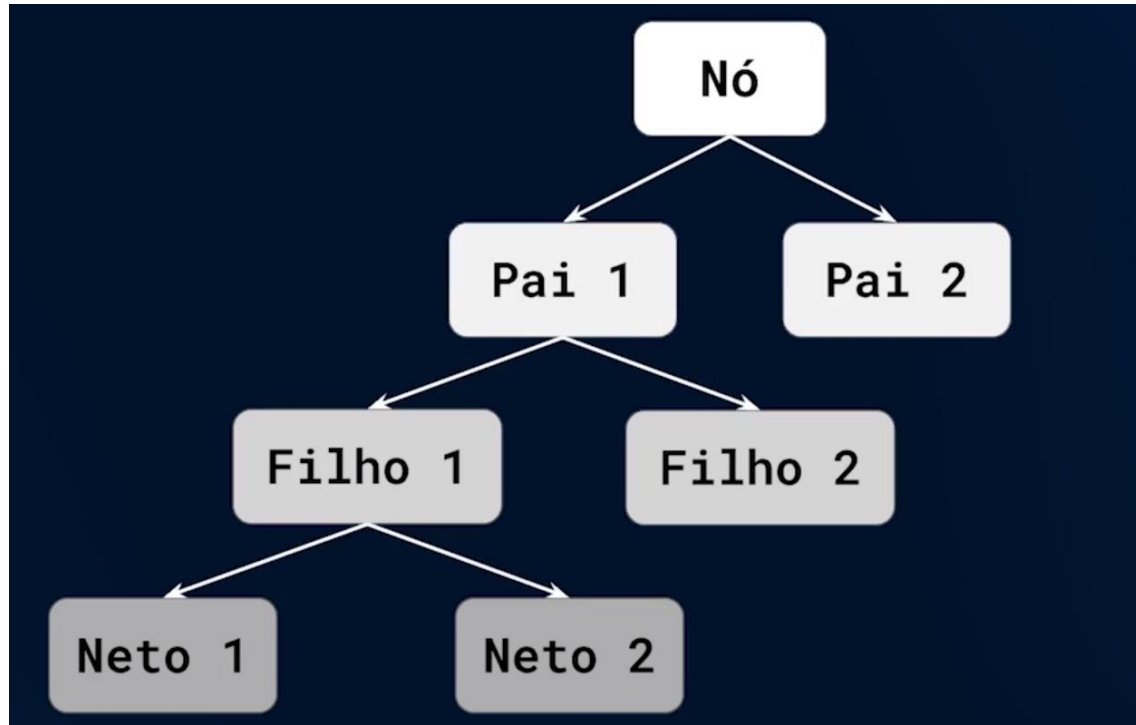
São como funções Javascript. Aceitam entradas chamadas props e retornam elementos React.

Permitem você dividir a interface em parte independentes, reutilizáveis e pensar em cada parte isoladamente.

Podem ser escritas no formato de **função ou de classe**.

Componentes de classes possuem recursos adicionais.

Componentes e Sub-componentes



Componentes

```
// componente de função
function Welcome(props) {
  return <Text>Olá, {props.name}</Text>;
}

// componente de classe
class Welcome extends React.Component{
  render(){
    return <Text>Olá, {this.props.name}</Text>
  }
}

// chamando o componente
export default function App() {
  return (
    <View>
      <Welcome name="Jackson"></Welcome>
    </View>
  );
}
```

Componentes Compostos

```
import React from 'react';
import { Text, View } from 'react-native';

function Welcome(props) {
  return <Text>Olá, {props.nome}</Text>
}
//posso chamar o mesmo componente várias vezes
export default function App() {
  return (
    <View >
      <Welcome name="Jackson"></Welcome>
      <Welcome name="Chiquinha"></Welcome>
      <Welcome name="Chaves"></Welcome>
    </View>
  );
}
```

Componentes Stateless e Stateful

Componentes Stateless:

- São componentes simples que **não possuem estado interno**.
- São funções que recebem *props* como entrada e retornam uma árvore JSX como saída
- São mais **performantes** e **fáceis de entender**, pois não possuem lógica complexa
- **Exemplo:** Um componente que apenas exibe um texto estático

Componentes Stateful:

- Possuem **estado interno** que pode ser modificado ao longo do tempo
- Usam classes para definir o estado e os métodos de atualização
- Possuem um ciclo de vida mais complexo, com métodos como *componentDidMount* e *componentWillUnmount*
- **Exemplo:** Um componente que exibe uma lista de itens que pode ser filtrada ou ordenada

Componentes Stateless e Stateful

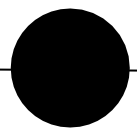
Componentes Stateless:

```
const MeuComponente = ({ texto }) => {  
  return <Text>{texto}</Text>;  
};
```

Componentes Stateless e Stateful

Componentes Stateful:

```
class MyComponent extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      count: 0,  
    };  
  }  
  render() {  
    return (  
      <div>  
        <Text>Contagem: {this.state.count}</Text>  
        <Button onPress={() => this.setState({ count: this.state.count + 1 })}>  
          Incrementar  
        </Button>  
      </div>  
    );  
  }  
}
```



Props

Props

São as **propriedades** do componente!

Um componente pode ou não ter props.

O React é bastante flexível, porém tem uma única regra:

Os componentes *não* devem alterar os valores de suas props (somente leitura).

```
function Welcome(props) {  
  return <Text>Olá, {props.nome}</Text>  
}
```

Props em componente de função.



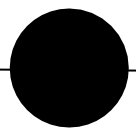
```
// componente de classe
class Welcome extends React.Component{
  render(){
    return <Text>Olá, {this.props.nome}</Text>
  }
}
```

Repare que quando criamos um componente utilizando classe:

- 1) Para acessar as propriedades do componente devemos utilizar `this.props`
- 2) **Deve-se obrigatoriamente sobrescrever o método `render`.**

Props em componente de classe.





State

State

O estado no React Native é um objeto que armazena dados que podem mudar ao longo do tempo

Ele é usado para controlar a interface do usuário do seu aplicativo, como mostrar ou ocultar elementos, alterar o texto de um botão ou atualizar a lista de itens em uma tela

Em componentes de classe (*Stateful*) são diferentes a sintaxe comparado com os componentes de função (*Stateless*)

Como usar os States

Como usar o estado:

1. **Defina o estado inicial:** O estado inicial é definido no construtor do componente
2. **Atualize o estado:** O estado pode ser atualizado usando o método `setState()`
3. **Renderizar o componente:** O componente será renderizado novamente sempre que o estado for atualizado

Usando os States

```
// exemplo com componente Stateless(Função)
import React, { useState } from 'react';
import {Text, View, Button } from 'react-native';

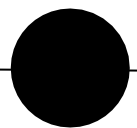
export default function App(){
  const [count, setCount] = useState(0); // método Get e Set do React

  const handleClick = () => {
    setCount(count + 1);
  };

  return (
    <View>
      <Text>Contagem: {count}</Text>
      <Button title="Clique aqui" onPress={handleClick} />
    </View>
  );
};
```

Usando os States

```
// exemplo com componente Stateful(Classe)
class App extends Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0,
    };
  }
  handleClick = () => {
    this.setState({ count: this.state.count + 1 });
  };
  render() {
    return (
      <View>
        <Text>Contagem: {this.state.count}</Text>
        <Button title="Incrementar" onPress={this.handleClick} />
      </View>
    );
  }
}
```



Style

Style

Não necessita de uma linguagem especial para estilizar o conteúdo. Isso é feito através do JavaScript.

Todos os componentes aceitam uma prop chamada “style”.

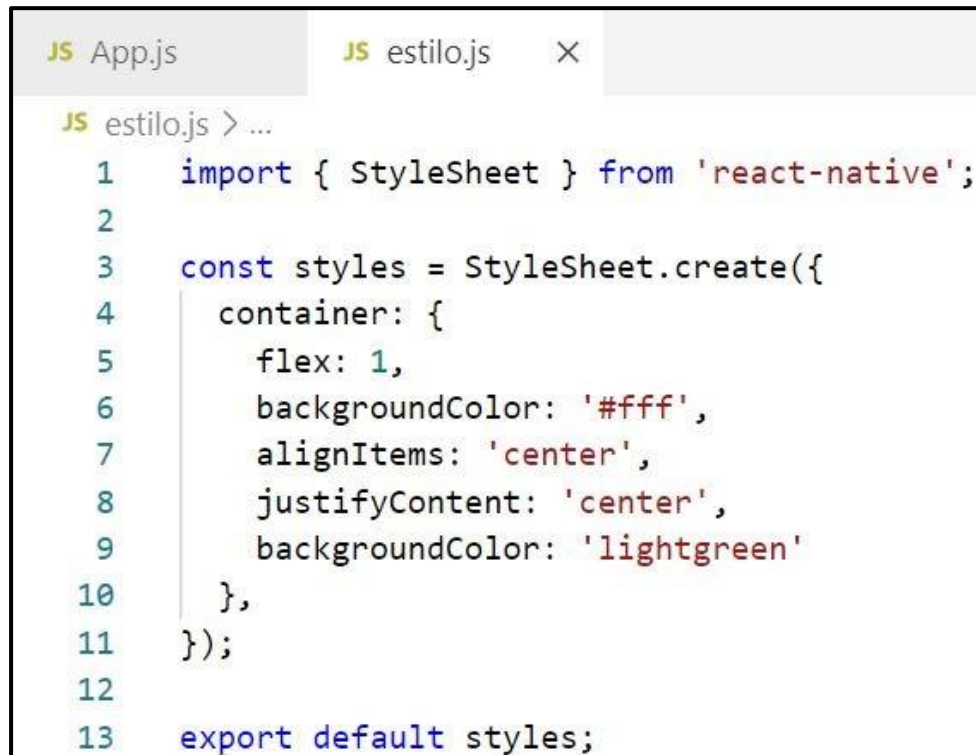
Sintaxe parecida com css, porém **utiliza camelCase.**

backgroundColor ao invés de background-color

Style

Pode-se fazer tudo no mesmo arquivo ou em arquivos separados.

Se optar por fazer em arquivos separados, deve-se exportar o estilo.



```
JS App.js JS estilo.js ×
JS estilo.js > ...
1  import { StyleSheet } from 'react-native';
2
3  const styles = StyleSheet.create({
4    container: {
5      flex: 1,
6      backgroundColor: '#fff',
7      alignItems: 'center',
8      justifyContent: 'center',
9      backgroundColor: 'lightgreen'
10   },
11 });
12
13 export default styles;
```

Style



**INSTITUTO
FEDERAL**
Santa Catarina

```
JS App.js  X  JS estilo.js

JS App.js > ...
1  import React from 'react';
2  import { Text, View } from 'react-native';
3  import styles from './estilo';
4
5  export default class App extends React.Component {
6      render(){
7          return(
8              <View style={styles.container}>
9                  <Text>Olá Mundo!</Text>
10             </View>
11         );
12     }
13 }
```

Instalando o React Native

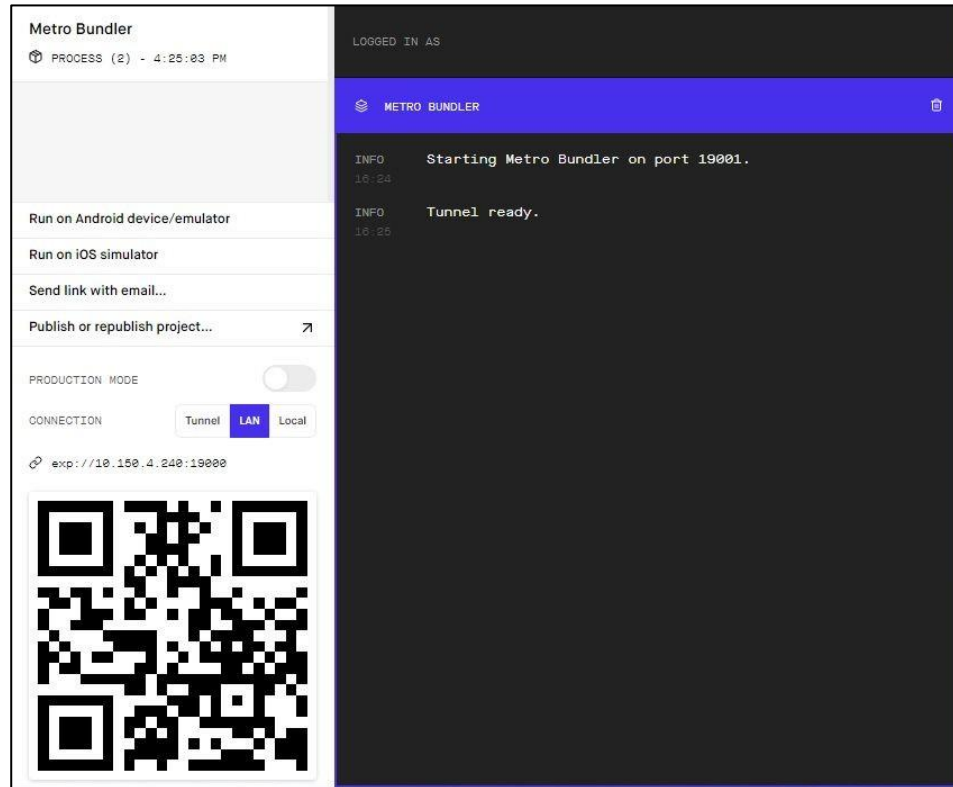
Para utilizar o React Native é preciso:

- Instalar o NodeJS ([https:// nodejs.org/en/](https://nodejs.org/en/))
- Instalar o Git ([https:// git-scm.com/](https://git-scm.com/))
- Instalar o VisualCode
- Instalar um emulador de sua preferência
 - Android Studio - Para computadores 8gb de Ram
 - LDPlayer - Mais Leve computados de 4gb.
 - Ou usar o proprio SmartPhone

Executando um App React Native

Para criar/ executar um app React Native é preciso:

- Navegar até a pasta onde deseja criar o projeto
- Cria um projeto *Expo*
 - *`npx create-expo-app@latest nomeProjeto`*
- Escolher a opção “blank”
- **`npx expo start` ou `yarn expo start`**



*Interface de conexão sendo executada no navegador
(<http://localhost:19002/>).*

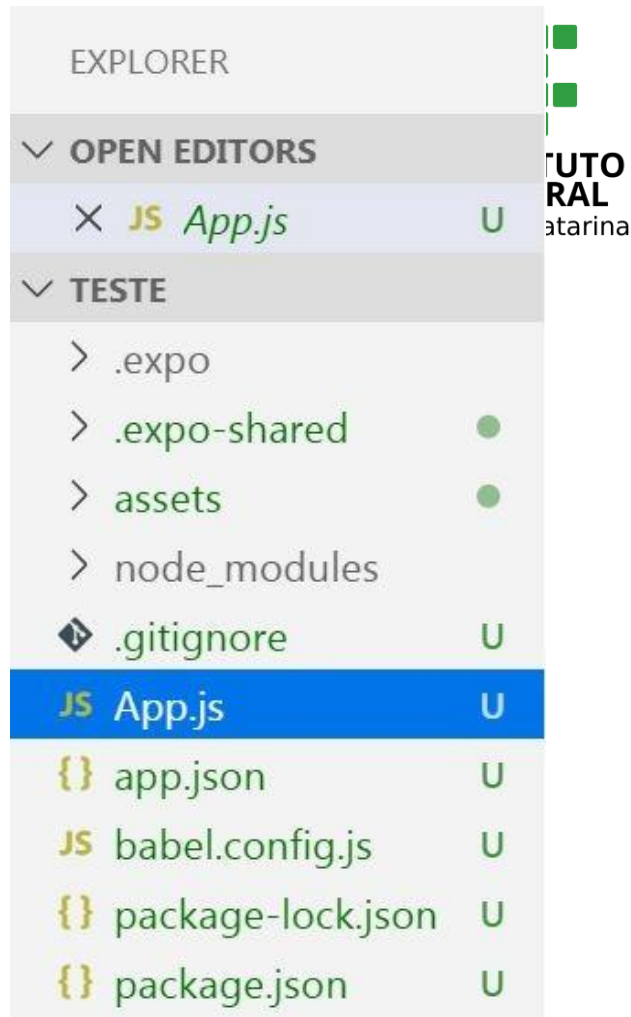
Open up App.js to start working on your app!

App react native sendo executado no emulador.

Editando a Aplicação

Para modificar a aplicação, vamos utilizar o Visual Studio Code. Basta abrir a pasta do projeto no editor.

O arquivo que nos interessa é o App.js.



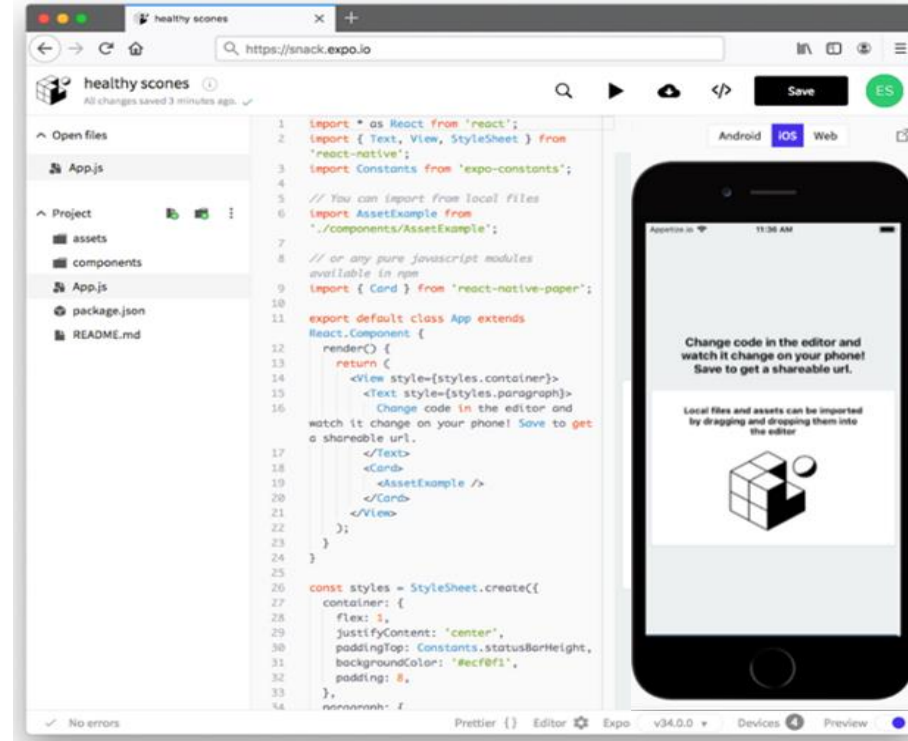
Emulador Online

Servem para simular um dispositivo real

Não precisa instalar

Emulador online no Navegador:

- Snack expo



```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Open up App.js to start working on your app!</Text>
      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```



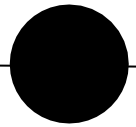
```
import { StatusBar } from 'expo-status-bar';  
import { StyleSheet, Text, View } from 'react-native';
```

```
export default function App() {  
  return (  
    <View style={styles.container}>  
      <Text>Open up App.js to start working on your app!</Text>  
      <StatusBar style="auto" />  
    </View>  
  );  
}
```

Componente

```
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    backgroundColor: '#fff',  
    alignItems: 'center',  
    justifyContent: 'center',  
  },  
});
```

Estilo



Olá Mundo!



```
import React from "react";
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Olá mundo!</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Hello World em React Native usando função.



```
import React from "react";
import { StyleSheet, Text, View } from "react-native";

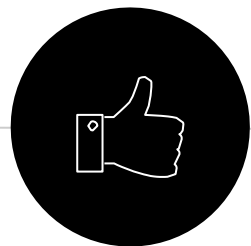
export default class App extends React.Component() {
  render() {
    return (
      <View style={styles.container}>
        <Text>Olá mundo!</Text>
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#fff",
    alignItems: "center",
    justifyContent: "center",
  },
});
```

Hello World em React Native usando classe.

Exercício Prático

1. Crie um componente que receba 3 propriedades por meio do *props* e exiba-os na tela.
2. Crie um componente de função que gere 6 números da sorte de forma aleatória para a mega-sena utilizando o *state*, os números gerados devem ser entre 1 a 99
3. Crie um componente de classe que utiliza o *componentDidMount*, carregue um componente chamado carro com um delay de 5 segundos, este componente deve trazer uma mensagem falando seu modelo, cor e placa. Em seguida, exiba também uma mensagem parabenizando o ganhador deste carro informando o seu número da sorte gerado de forma aleatória entre 1000 a 5000



Obrigado!

Dúvidas ?

Você pode me encontrar em:

- jackson.meires@ifsc.edu.br



@jacksonm